

Name: Asad
Class: BSAI 4C
Roll Number: 141

SMART VOLUNTEER DISPATCH

Instructor: Rasikh Ali

Project Description

This is an intelligent ticketing system that automatically assigns the right volunteer to a community emergency. When a dispatcher enters a text description and a location, the system uses **Natural Language Processing (NLP)** to classify the emergency type. Then a **Prolog knowledge base** applies logic rules to find the most suitable available volunteer based on skills, location proximity, and availability. The result is served through a **Flask web application**.

Combination Used

NLP (Machine Learning) + Logic Programming (Prolog) + Flask Web Framework

- **NLP:** TF-IDF vectorizer + Multinomial Naïve Bayes classifier to predict emergency category from description.
- **Prolog:** Knowledge base of volunteers with rules that match the predicted category and location.
- **Flask:** Web interface for user input and display of assignment results.

Project Structure

```
volunteer_dispatch/  
├──  
├── app.py                # Main Flask application  
├── train_emergency_model.py # Script to train the NLP model  
├── volunteer_rules.pl     # Prolog knowledge base  
├── models/  
│   ├── emergency_model.pkl  
│   └── vectorizer.pkl  
├── templates/  
│   └── index.html  
└── static/  
    └── style.css
```

1. Training the NLP Model (train_emergency_model.py)

```

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
import joblib

# Load dataset (description, category)
data = pd.read_csv('emergency_data.csv')

X = data['description']
y = data['category']

vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)
X_vec = vectorizer.fit_transform(X)

model = MultinomialNB()
model.fit(X_vec, y)

joblib.dump(model, 'models/emergency_model.pkl')
joblib.dump(vectorizer, 'models/vectorizer.pkl')
print("Model saved.")

```

2. Prolog Knowledge Base (volunteer_rules.pl)

```

% volunteer_rules.pl
% Volunteer facts: id, name, skill list, location, available (1=yes)
volunteer(1, 'Ali', [medical, first_aid], 'BlockA', 1).
volunteer(2, 'Sara', [fire_safety, rescue], 'BlockB', 1).
volunteer(3, 'Ahmed', [food_prep, logistics], 'BlockC', 0).
volunteer(4, 'Fatima', [medical, shelter], 'BlockA', 1).
volunteer(5, 'Bilal', [flood_rescue, driving], 'BlockD', 1).

% Skills needed per emergency type
required_skill(medical, medical).
required_skill(medical, first_aid).
required_skill(fire, fire_safety).
required_skill(flood, flood_rescue).
required_skill(food_distribution, food_prep).
required_skill(food_distribution, logistics).

% Same location means near
near_location(Loc, Loc).

% Main matching rule
assign_volunteer(EmergencyType, Location, Id, Name) :-
    required_skill(EmergencyType, Skill),
    volunteer(Id, Name, Skills, VolLocation, 1),
    member(Skill, Skills),
    near_location(Location, VolLocation).

% Helper: member/2
member(X, [_|_]).
member(X, [_|T]) :- member(X, T).

```

3. Flask Application (app.py)

```

from flask import Flask, render_template, request
import joblib
from pyswip import Prolog

app = Flask(__name__)

# Load NLP model
model = joblib.load('models/emergency_model.pkl')
vectorizer = joblib.load('models/vectorizer.pkl')

# Connect to Prolog
prolog = Prolog()
prolog.consult('volunteer_rules.pl')

def assign_volunteer(emergency_type, location):
    """Query Prolog for best volunteer."""
    query = f"assign_volunteer({emergency_type}, {location}, Id, Name)"
    try:
        result = list(prolog.query(query))
        if result:
            return result[0]['Id'], result[0]['Name']
        return None, None
    except Exception as e:
        print(e)
        return None, None

@app.route('/', methods=['GET', 'POST'])
def index():
    assignment = None
    if request.method == 'POST':
        description = request.form['description']
        location = request.form['location']

        # NLP classification
        X_vec = vectorizer.transform([description])
        category = model.predict(X_vec)[0]

        # Prolog volunteer assignment
        vol_id, vol_name = assign_volunteer(category, location)

        assignment = {
            'category': category,
            'vol_id': vol_id,
            'vol_name': vol_name if vol_name else 'No available volunteer',
            'description': description,
            'location': location
        }
    return render_template('index.html', result=assignment)

if __name__ == '__main__':
    app.run(debug=True)

```

4. Frontend Template (templates/index.html)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Smart Volunteer Dispatch</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
">
</head>
<body>
  <div class="container">
    <h1> Smart Volunteer Dispatch</h1>
    <p>Report an emergency to dispatch the right volunteer.</p>
    <form method="post">
      <label>Emergency Description:</label><br>
      <input type="text" name="description" placeholder="e.g., a person
needs medical help" required>
      <br><label>Location (Block):</label><br>
      <input type="text" name="location" placeholder="BlockA" required>
      <br>
      <button type="submit">Dispatch Volunteer</button>
    </form>

    {% if result %}
    <div class="result">
      <h2>Dispatch Result</h2>
      <p><strong>Emergency Category:</strong> {{ result.category }}</p>
      <p><strong>Assigned Volunteer:</strong> {{ result.vol_name }}</p>
      {% if result.vol_id %}
      <p><strong>Volunteer ID:</strong> {{ result.vol_id }}</p>
      {% endif %}
      <p><em>Description:</em> {{ result.description }}</p>
      <p><em>Location:</em> {{ result.location }}</p>
    </div>
    {% endif %}
  </div>
</body>
</html>

```

5. Styling (static/style.css)

```

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #eef2f5;
  margin: 0;
  padding: 2rem;
  display: flex;
  justify-content: center;
}
.container {
  background: white;
  padding: 2rem;
  border-radius: 10px;
  max-width: 600px;
  width: 100%;
  box-shadow: 0 0 15px rgba(0,0,0,0.1);
}
input[type="text"] {
  width: 80%;
  padding: 0.6rem;
  margin: 0.5rem 0 1rem;
  border: 1px solid #ccc;
  border-radius: 5px;
}
button {
  padding: 0.7rem 1.4rem;
  background: #2c3e50;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
button:hover {
  background: #1a252f;
}
.result {
  margin-top: 2rem;
  padding: 1rem;
  background: #f0f8ff;
  border-left: 4px solid #2c3e50;
}

```

How to Run

1. Create `emergency_data.csv` with columns `description` and `category`.
2. Run `python train_emergency_model.py` to build and save the model.
3. Start the Flask server: `python app.py`.
4. Open `http://127.0.0.1:5000`, enter an emergency description and location, then dispatch.

— End of Project —